



... to this, at the click of a button

general idea, we narrow down the workings of Gecko and WebKit (as used by the Chromium base) to two distinct points – the process architecture and extensibility.

The Gecko process architecture is such that it handles all of the concurrent tabs on a single process thread. While this results in a comparatively slower interface, in the long term, it pays off to not fork a new process for every little task that needs to be accomplished – the model commonly used under the WebKit architecture. What this means is that while the first few tabs with WebKit would seem faster, it's because the engine is only utilising a few extra resources from the computer to parallelise the work and bring with it the overhead involved, but when you go up to double digits in tabs opened, with some of them doing resource-intensive work, such as playing videos or working with comprehensive web-apps, this model returns to cripple the overall performance of the system, and by extension, of the browser itself, due to the sheer number of the processes that are now being requested to be forked and maintained by the system. Firefox, as has often been cited, tends to do much better at handling such high pressure loads, because all of the browser's workings are restricted to a single thread, which is much more maintainable at a system level. The difference in the memory